

idTech4 Materials

Blender 4.x / 5.x .mtr Material Generation Add-on

Version 1.0.0 · Author: Samson & Claude

1. Overview

The idTech4 Materials add-on builds real, fully textured Blender materials from idTech 4 .mtr material declarations. It reads Doom 3, Doom 3 BFG, Quake 4 (including its guide macro layer), Prey and The Dark Mod from one merged keyword table — there is no per-game mode switch to set.

It is both a panel you use directly and the engine the other idTech4 add-ons call: when you tick Generate Materials in a model or map import dialog, that import calls straight into this add-on. Everything the panel can do, an import can do, with the same settings.

Supported operations:

- Build materials for every material in the scene, every material declared in the source, or just the materials on the selected objects
- Four fidelity modes, from an engine-exact reproduction of the ARB2 renderer down to a single diffuse texture
- Three parameter policies, deciding what happens to time, parm, global, sound and table expressions
- Editor-texture management — find, fade and hide the materials that are nothing but a level-editor placeholder image
- Shader parm sliders that materials respond to live
- A build report, a list of the tables that were created, and a node-graph topology census

2. Installation

1. Open Blender and navigate to Edit > Preferences > Add-ons.
2. Click Install... and select idTech4_material_import.py.
3. Tick the checkbox next to "idTech4 Materials".
4. Close Preferences. The add-on is now active.

After installation the following entries appear:

- 3D Viewport > N-Panel > idTech4 > Sources
- 3D Viewport > N-Panel > idTech4 Mtr > Materials

NOTE: This add-on keeps its own copy of the shared Sources panel, and also shows the same three paths at the top of its own Materials panel with browse and clear buttons beside them. Either place edits the same shared config file.

3. Sources

idTech 4 games do not ship their assets as loose files. Doom 3, Quake 4, Prey and The Dark Mod pack them into .pk4 archives; Doom 3 BFG packs them into .resources files. This add-on does not read inside either. It reads an ordinary directory tree on disk, and nothing else.

So before you set Base Directory, extract everything. A .pk4 is an ordinary zip archive — any zip tool will open one, and most will do it without renaming the file first. A .resources file is not a zip and needs a Doom 3 BFG extraction tool.

Extract every archive into one folder, preserving the paths stored inside them, so the result is the full idTech 4 directory structure:

```
<game>/
  base/                ← Base Directory points HERE
    def/               entityDef and model declarations
    dds/               precompressed textures
    guis/
    maps/              .map source files
    materials/         .mtr material declarations
    models/            .md5mesh, .md5anim, .ase, .lwo
    skins/             .skin declarations
    textures/          .tga / .png / .jpg source textures
    generated/         BFG only: .bimage, .bmd5mesh,
                      .bmd5anim, .base, .blwo caches
```

IMPORTANT: Base Directory is the "base" folder ITSELF — the one that CONTAINS materials/, models/, textures/ and the rest — not the game folder above it, and not one of the folders inside it. Every path in a .mtr, .def, .skin or .map is written relative to this folder, so pointing one level too high or too low resolves nothing.

NOTE: A game ships several numbered archives (pak000.pk4, pak001.pk4, ... or game00.pk4, game01.pk4, ...) and the later ones patch the earlier ones. Extract them in ascending numeric order and let later files overwrite earlier ones — that is the same precedence the engine applies at run time. Extracting them in the wrong order silently leaves you with pre-patch assets.

For a mod or an expansion, extract its own archives into its own folder beside base — d3xp, darkmod, and so on — and set Mod Base Directory to that folder while leaving Base Directory on base. A mod supplies part of the tree and inherits the rest, and the add-ons search the two the same way the engine does.

NOTE: The extracted tree only has to contain what you actually intend to load. Sound, video and script archives can be left packed. The folders that matter are the ones in the diagram above.

The add-on needs to be told where the .mtr declarations and the textures they name live. Three paths, all stored in a small JSON file under Blender's per-user config directory and shared with every other idTech4 add-on.

Setting	Required	Purpose
Base Directory	Yes	A standard Doom 3-style "base" folder. Textures are resolved under it, and its own materials/ subfolder is the default .mtr source.
Mod Base Directory	No	An optional second tree searched BEFORE Base Directory, with Base Directory still searched behind it — the engine's fs_game behaviour.
Materials Source	No	One specific .mtr file, or a folder of them. When set, it is used on its own and nothing else is searched. Leave it blank and the materials/ subfolder under Mod Base (if set), then Base Directory, is used.

The path fields themselves are read-only text: they mirror the shared config rather than a scene property, so a typed path would be silently discarded. Use the folder button to browse and the X to clear.

Textures are resolved with the engine's own extension search, .dds included. For The Dark Mod this matters a great deal — around 99% of its textures exist only under dds/, so a resolver that skips that tree leaves most Dark Mod materials pink.

4. Generating Materials

3D Viewport > N-Panel > idTech4 Mtr tab > Materials panel. Set what to build, how faithfully to build it, and press Generate Materials.

4.1 What to Build

Option	Type	Default	Description
Create	Enum	All Scene Materials	All Scene Materials: every material already present in this .blend. All Source Materials:

Option	Type	Default	Description
			every material declared anywhere under the Materials Source. Selected Objects' Materials: only the materials used by the selected objects.

IMPORTANT: All Source Materials on a full game tree is tens of thousands of materials. It is there for corpus work, not for ordinary use — build what your scene actually needs.

4.2 Fidelity Modes

Four modes, each justified by a measurement rather than by taste. The panel shows the measured speed-up under the dropdown as you change it.

Mode	What it builds
Maximum	The ARB2 renderer's own draw sequence: one Principled BSDF per flushed interaction pass, engine-exact specular, cube maps, and every ambient stage composited through its real OpenGL blend equation. The reference rung, and the most expensive.
Good	Measured 1.14× faster than Maximum on real Doom 3 materials. Collapses the engine's interaction passes into one and uses a Diffuse BSDF; the pass collapse is worth 1.77× on a material that flushes three of them, but almost every real material flushes one. Keeps normals, heightmaps, cube maps, the roughness estimate and full ambient compositing. What it gives up is the specular highlight.
Basic	Measured 1.30× faster than Maximum, and 91 MB lighter than Good across 300 materials: it loads no specular texture at all, and 16,593 corpus materials ship one. Builds only the ambient stages that carry alpha — decals, overlays and cutouts — and caps the stack at four.
Simple	Measured 1.50× faster than Maximum and a third of its resident texture memory (115 MB against 338 MB over 300 materials): the diffuse texture on a lit Diffuse BSDF, nothing else. Lit rather than emissive on purpose — it measured the same, and geometry placement reads correctly against scene lighting, which is the point of this mode.

NOTE: Ambient stage count is the other half of the cost. Four additive stages measured +31% render time, and Prey ships materials 200 stages deep, which is why every mode below Maximum caps the stack.

4.3 Parameters

An idTech4 material's expressions can reach time, parm0..11, global0..7, a sound amplitude and arbitrary table lookups. What to do with them is often a bigger performance lever than a whole fidelity mode.

Policy	Effect
Baked (default)	Fold every expression once, now, against the current frame and the slider values below. No drivers at all, so nothing re-runs per frame and nothing depends on the driver namespace surviving a file load. The mode to use for a whole-map import. Moving a slider afterwards needs a rebuild.
Dynamic	Drivers for expressions that reach `time`, and a recorded expression re-folded on slider moves for the rest. The material animates with the timeline and responds to the sliders. Costs real frame time: on mars_city1, 219 drivers were 74 ms of a 91 ms frame — and evaluating all 219 expressions was 0.3 ms of that, the rest being EEVEE rebuilding each material's GPU shader.
Skip	Refuse every parameter. Conditional stages are dropped without being evaluated and dynamic terms use their neutral defaults. The simplest and cheapest result, and the least faithful.

IMPORTANT: Dynamic materials depend on values Blender keeps only for the current session. Reopening a .blend does not restore them by itself; the add-on reinstalls them on file load, which is why it must be enabled when you open a file containing dynamic materials. Baked materials have no such dependency.

4.4 Build Options

Option	Type	Default	Description
Ambient Cap	<i>Int</i>	-1	Maximum ambient stages built per material. 0 means uncapped; -1 uses the fidelity mode's own value (Maximum uncapped, Good 8, Basic 4, Simple 2).
Prefer Editor Image	<i>Bool</i>	Off	Simple mode only: draw the qer_editorimage instead of the diffuse stage. Cheaper to load and closer to what the level editor shows. Greyed out outside Simple mode.
Skip Light Materials	<i>Bool</i>	Off	Do not build light shaders at all. They are not surface shaders, so what gets built is only a preview of the projection texture.
Lights Use Editor Image	<i>Bool</i>	Off	Preview a light shader with its qer_editorimage rather than its projection texture. Off by default because 1,267 of the corpus's 1,271 light materials have a stage map and only 15 have an editor image.

Option	Type	Default	Description
			Greyed out while Skip Light Materials is on.

4.5 Running the Build

Press Generate Materials. The add-on reloads the .mtr database, works out which material names are in scope, builds them, refreshes the Editor Textures list, and writes a full report to a text datablock whose name the status line gives you.

If nothing is built, the two usual reasons are reported directly: no .mtr source could be resolved (set a Base Directory or a Materials path), or nothing was in scope for the current Create setting.

5. Panel Sections

Under the Generate Materials button sit five collapsible sections, all closed by default. Each header carries a count so you can see whether it is worth opening.

5.1 Editor Textures

Many idTech4 materials are nothing but a `qer_editorimage` — a placeholder the level editor draws so a brush is visible, with no real in-game surface behind it. Trigger volumes, clip brushes, monster clip, AAS boundaries and visportals are all of this kind. Imported into Blender they are opaque boxes sitting in front of the geometry you wanted to look at.

This section finds them and lets you fade or hide them, individually or all at once, without deleting anything.

Control	Type	Default	Description
Global Visibility	<i>Bool</i>	On	Show or hide every editor-texture object at once.
Global Opacity	<i>Float</i>	1.0	Opacity of every editor-texture material at once. Fade them back rather than hiding them outright when you still want to see where they are.
Split Mixed-Material Objects	<i>Bool</i>	Off	Split the faces using an editor-texture material off into their own ' <code><object>_edittex</code> ' object, so they can be hidden independently. Off by default: it edits geometry.
List	<i>UList</i>	—	Every editor-texture material, each with its own eye toggle independent of Global Visibility.
Refresh Editor Textures	<i>Button</i>	—	Rescan every material in the file and rebuild the list.

Control	Type	Default	Description
Add Selected Material	<i>Button</i>	—	Add the active material on the selected object to the list, splicing in the node setup the opacity and visibility toggles need. Whatever the material already renders is kept as the visible side of the mix, so nothing is discarded.

5.2 Shader Parmes

idTech4 sets parm0..11 per entity at run time and global0..7 renderer-wide, and there is no way to know either from a .mtr. These sliders are the preview values a static import uses instead. parm0..3 are the entity colour and default to opaque white.

Control	Type	Default	Description
parm0 ... parm11	<i>Float</i>	1,1,1,1 then 0	The per-entity shader parms.
global0 ... global7	<i>Float</i>	0.0	The renderer-wide shader parms.
Sound	<i>Float</i>	0.0	The `sound` expression term — an entity's current sound amplitude. Roughly 500 light materials modulate their brightness with it.
Spectrum	<i>Int</i>	0	0 shows the ordinary world; N also reveals materials declaring `spectrum N`. Only 45 materials in the whole corpus, but they are invisible without it.
Refresh Parameters	<i>Button</i>	—	Re-fold every socket that records a parm/global/sound expression. Slider moves do this automatically; the button is for a value changed from a script.

NOTE: Under Dynamic, moving a slider updates the affected materials immediately. Under Baked the section header says "(rebuild required)" and the sliders set the values the NEXT build will fold in. Under Skip the whole section is disabled.

5.3 Created Tables

A .mtr `table` declaration is a named curve materials index into. Any table a generated driver looks up is registered and listed here, with a button to print its contents. When nothing in the scene looks up a table, the section says so.

5.4 Material Topologies

Press Count Topologies to group every idTech4 material in the scene by the structure of its node graph. The summary gives the number of distinct topologies across the number of materials, and the list shows each signature with an example material and its node count. This

is the quick way to see whether a scene's materials are as varied as they look, and to find the outlier that is much heavier than the rest.

5.5 Report

The header carries the row count, and the summary line is drawn in the body. Open Full Report opens the untruncated report in its own Text Editor window — the findings live in the text datablock rather than in a fixed-height sidebar list, which could never show a whole run's worth of them without truncating every line to the panel's width.

A clean run says so explicitly rather than showing an empty list, so "nothing to report" and "nothing has been run yet" are never confused.

6. What the Parser Covers

The .mtr parser is written to mirror the engine function by function, so the two can be diffed by eye, and it is regression-tested against the shipping corpora of all five supported games. It has no dependency on Blender at all, which is what lets those tests run without launching it.

Area	Coverage
Lexing and decl scanning	idLexer's own tokenisation rules, including the guide macro preprocessing pass Quake 4 needs.
Image programs	heightmap, addnormals, add, smoothnormals, invertalpha, invertcolor, makeintensity, makealpha, scale and the rest of R_ParseImageProgram_r, nested as the engine nests them.
Expressions	The full expression grammar, including table lookups, conditionals, and every dynamic term.
Stages	Blend modes resolved to their real OpenGL blend equations, alpha test, vertex colour modes, per-stage texture matrices (scroll, translate, scale, centerScale, shear, rotate), clamp and repeat modes, and cube maps.
Material post-pass	Coverage and sort resolution exactly as idMaterial::Parse does it — including the detail that the LAST coverage write wins, which matters for the ninety-odd materials that put an alphaTest stage after `translucent`.
Decl names	Canonicalised the way idDeclManagerLocal::MakeNameCanonical does it: backslashes to slashes, lowercased, truncated at the last dot, on both registration and lookup.

NOTE: In idTech 4, `translucent` means additive interaction passes, not alpha blending. The generated materials reflect that, which is why a translucent material can look brighter rather than see-through.

An undeclared material name is not an error: the engine treats it as a texture path and generates a material from it implicitly, and so does this add-on. Diagnostics about genuinely unknown keywords are reported best-effort — a number of them are materials that are simply broken in the shipping games.

7. Working With the Other Add-ons

Every idTech4 importer that can build materials calls this add-on to do it, passing the same two settings the panel exposes:

Caller	Material Mode	Parameters
MD5 mesh import	Maximum	Dynamic
.ase / .lwo import	Maximum	Dynamic
.map import	Simple	Baked
Materials panel	Good	Baked

The defaults differ on purpose. One model's worth of materials is cheap enough to build at full fidelity and to give live drivers to; a whole level's worth is not. After an import, the panel's own settings are synced to whatever that import used, so pressing Generate Materials again does the same thing rather than something surprising.

An import that builds materials also writes its build report through this add-on, scoped to the materials that import actually built rather than to the whole .mtr tree, and names the resulting text datablock in its status line.

NOTE: If this add-on is not installed and enabled, every other add-on's material option is greyed out with a message saying so. The imports still work; the geometry simply gets blank placeholder materials.

8. Tips and Troubleshooting

Nothing Resolves At All

If every lookup fails — no .mtr files found, no models found, no .def files found — the usual cause is that the game's assets are still inside their .pk4 or .resources archives. These add-ons cannot read into an archive; everything has to be extracted to a real directory tree first. The second most common cause is Base Directory pointing at the game folder rather than at the base folder inside it: it must be the folder that directly contains materials/, models/ and textures/.

Everything Is Pink

Pink means the texture file was not found, not that the material failed. Check Base Directory, and check whether the game keeps its textures compressed — The Dark Mod keeps almost all of them under dds/ only.

Materials Are Flat White

Flat white means the material DECL was not found and a blank placeholder was used. Read the report: it names each one. The usual cause is a Materials Source pointed somewhere that does not contain the game's .mtr files, or a mismatch between how a model spells a material name and how the .mtr declares it — which the canonical-name matching is there to absorb, backslashes and forward slashes alike.

The Viewport Is Slow After a Map Import

Drop Material Mode to Simple and Parameters to Baked and rebuild. Parameters is usually the bigger of the two: a scripted driver runs on every frame change whether or not anything about it changed.

Animated Materials Stopped Animating After Reopening the File

Dynamic materials need this add-on enabled when the file is opened; the values their drivers read do not survive a file load on their own and the add-on reinstalls them. If they are still static, rebuild with Dynamic, or switch to Baked if you do not need the animation.

A Trigger Volume Is Blocking the View

Open Editor Textures, press Refresh, and either turn Global Visibility off or pull Global Opacity down. If the offending faces share an object with real geometry, tick Split Mixed-Material Objects first — but note that one does edit the mesh.

Rebuilding After Changing a Slider

Only Dynamic materials follow the sliders live. Under Baked, move the sliders first, then press Generate Materials.

Reopening the Report

The report is an ordinary text datablock. Open any Text Editor and pick it from the datablock browser, or press Open Full Report again.